deleted later. The following listing shows a full implementation of `pop()` using such a scheme.

Listing 7.6. An implementation of `pop()` using hazard pointers

```
std::shared_ptr<T> pop()
{
    std::atomic<void*>& hp=get_hazard_pointer_for_current_thread();
    node* old_head=head.load();
    do
    {
        node* temp;
        do                                          // 1 Loop until you've set the hazard pointer to head
        {
            temp=old_head;
            hp.store(old_head);
            old_head=head.load();
        } while(old_head!=temp);
    }

    while(old_head &&
          !head.compare_exchange_strong(old_head,old_head->next));
    hp.store(nullptr);                              // 2 Clear hazard pointer once you're finished
    std::shared_ptr<T> res;
    if(old_head)
    {
        res.swap(old_head->data);
        if(outstanding_hazard_pointers_for(old_head))   // 3 Check for hazard pointers referencing a node before you delete it
        {
            reclaim_later(old_head);                // 4
        }
        else
        {
            delete old_head;                        // 5
        }
        delete_nodes_with_no_hazards();             // 6
    }
    return res;
}
```

First off, you've moved the loop that sets the hazard pointer *inside* the outer loop for reloading `old_head` if the compare/exchange fails ❶. You're using `compare_exchange_strong()` here because you're actually doing work inside the `while` loop: a spurious failure on `compare_exchange_weak()` would result in resetting the hazard pointer unnecessarily. This ensures that the hazard pointer is correctly set